DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 03
Spring Boot 4, setup e fluxo Git

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Nesta semana o ServiceHub vira código. Antes de gerar o projeto, vale entender as ideias que o Spring usa o tempo todo: inversão de controle, anotações, pacotes e camadas. Depois vem o Spring Boot 4, com seus starters e sua configuração externa, e o primeiro endpoint, `GET /api/v1/info`. A segunda metade trata do processo: Git, GitHub Flow, Conventional Commits, versionamento e tags. Por fim, dois documentos que a IA não adivinha: o arquivo de contexto do projeto e o primeiro registro de decisão de arquitetura (ADR).

Conteúdo

1. Objetivos da semana
2. Biblioteca ou framework
3. Anotações Java
4. Pacotes e visibilidade
5. Arquitetura em camadas
6. O contêiner do Spring
7. Configuração externa
8. Starters e o Initializr
9. Git: controle de versão distribuído
10. GitHub Flow, commits e versões
11. Contexto para a IA e ADRs
12. Aplicação no ServiceHub
13. Construindo com IA
14. Glossário
15. Exercícios
16. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. explicar inversão de controle, o papel das anotações e como o Spring cria e injeta beans;
2. organizar o código em pacotes e camadas, com a visibilidade mais restrita possível;
3. gerar um projeto Spring Boot 4 com Java 21, escolher os starters certos e externalizar a configuração;
4. trabalhar com Git e GitHub Flow: branch, commit no padrão Conventional Commits, pull request e tag;
5. escrever o arquivo de contexto do projeto para a IA e um ADR no formato de Nygard.

2. Biblioteca ou framework

↑ conteúdo

Uma **biblioteca** é código que você chama quando precisa. Um **framework** é uma estrutura pronta que chama o *seu* código na hora certa. Essa troca de quem está no comando se chama **inversão de controle** (*IoC*, de *Inversion of Control*), resumida no lema "não nos chame, nós chamamos você".

Aspecto	Biblioteca	Framework
Quem controla o fluxo	O seu programa	O framework
Como você usa	Chama uma função ou cria um objeto	Escreve classes que ele encontra e executa
Exemplo genérico	`java.time` para calcular datas	Um servidor web que chama o seu método a cada requisição
No ServiceHub	Jackson converte um objeto em JSON	O Spring chama `InfoController.info()` em `GET /api/v1/info`

O código que chama `info()` não está no projeto: está no Spring MVC. O preço é aprender as convenções do framework; o ganho é não reescrever servidor, roteamento e conversão de JSON.

3. Anotações Java

↑ conteúdo

Uma **anotação** é um metadado preso a uma classe, método, campo ou parâmetro, escrito com `@`. Ela não muda o código por si só: alguém precisa lê-la. Pode ser o compilador, uma ferramenta de build ou o framework, em tempo de execução, por **reflexão** (a capacidade de um programa Java inspecionar as próprias classes enquanto roda).

Duas meta-anotações definem uma anotação nova: `@Target` diz onde ela pode ser usada, e `@Retention` diz até quando ela sobrevive.

Retenção	Onde a anotação existe	Exemplo
SOURCE	Só no código-fonte; o compilador descarta	`@Override`
CLASS	No `.class`, mas não em execução (é o padrão)	uso de ferramentas
RUNTIME	Em execução, lida por reflexão	`@RestController`

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Auditado {          // declara a anotação
    String valor() default "";
}

@Auditado(valor = "pedidos")         // usa a anotação
public class PedidoService { }
```

O Spring só enxerga `@RestController`, `@Service` ou `@GetMapping` porque elas têm retenção RUNTIME. Anotações também podem compor outras: `@RestController` é, ela mesma, anotada com `@Controller` e `@ResponseBody`.

4. Pacotes e visibilidade

↑ conteúdo

Um **pacote** agrupa classes relacionadas e cria um espaço de nomes: `com.servicehub.controller.InfoController` não se confunde com outra `InfoController` de outro pacote. Por convenção, o nome começa pelo domínio invertido da organização, em minúsculas. Junto com os modificadores de acesso, o pacote decide quem pode usar o quê:

Modificador	Mesma classe	Mesmo pacote	Subclasse em outro pacote	Qualquer lugar
`private`	sim	não	não	não
sem modificador (pacote)	sim	sim	não	não
`protected`	sim	sim	sim	não
`public`	sim	sim	sim	sim

Regra prática: comece pelo acesso mais restrito e abra só quando precisar. No ServiceHub, o conjunto `SORTABLE` de cada controller não tem modificador: só o pacote `controller` o enxerga.

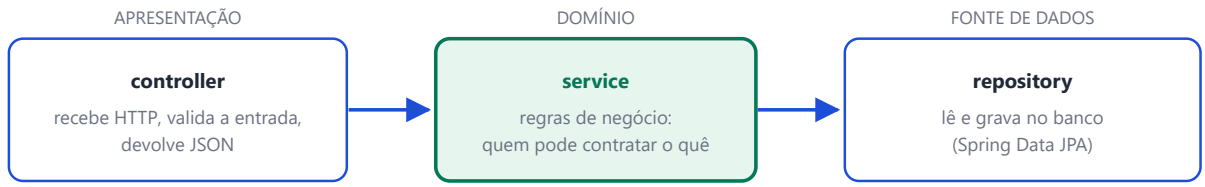
CONFUSÃO COMUM

O pacote também importa para o Spring. A varredura de componentes começa no pacote da classe anotada com `@SpringBootApplication` e desce pelos subpacotes. Uma classe colocada fora dessa árvore, por exemplo em `com.outra.coisa`, simplesmente não é encontrada, e o endpoint responde 404 sem nenhum erro na inicialização.

5. Arquitetura em camadas

↑ conteúdo

Separar o sistema em **camadas**, cada uma com uma responsabilidade, permite trocar ou testar uma parte sem mexer nas outras. Fowler descreve três camadas principais: apresentação, domínio e fonte de dados. Exemplo genérico, uma loja virtual: a tela do carrinho é apresentação, a regra do frete é domínio, a tabela de pedidos é fonte de dados.



As três camadas do ServiceHub e o sentido das dependências.

A **regra da dependência** diz que as setas só apontam num sentido. O service não conhece status codes; o repository não decide regra de negócio. Na Arquitetura Limpa, de Robert C. Martin, a ideia vai além: as dependências apontam para as regras de negócio, e o framework e o banco são detalhes.

Regra no controller

"Ninguém contrata o próprio serviço" escrita no controller. Quando surgir um segundo ponto de entrada (uma fila, uma tarefa agendada), a regra precisa ser copiada, e uma das cópias vai ficar para trás.

Regra no service

A regra mora no service. Qualquer ponto de entrada chama o mesmo método, e um teste de unidade verifica a regra sem subir HTTP.

6. O contêiner do Spring

↑ conteúdo

O Spring mantém um **contêiner IoC**, o `ApplicationContext`, que cria, liga e destrói objetos por você. Cada objeto gerenciado por ele se chama **bean**. É a inversão de controle aplicada à criação de objetos: você declara do que precisa; o contêiner decide quando e como criar.

Ciclo de vida de um bean

1. **Varredura:** na subida, o Spring procura classes anotadas nos subpacotes da aplicação.
2. **Criação:** chama o construtor e injeta as dependências, que também são beans.
3. **Inicialização:** roda métodos anotados com `@PostConstruct`, se houver.
4. **Uso:** o bean atende todas as requisições enquanto a aplicação está no ar.
5. **Destruição:** no desligamento, roda métodos com `@PreDestroy`.

CONFUSÃO COMUM

Por padrão, o bean é **singleton**: uma única instância, compartilhada por todas as requisições ao mesmo tempo. Se dois clientes chamam o mesmo endpoint simultaneamente, existe um só `InfoController`. Por isso nunca se guarda dado de um usuário num campo do controller ou do service: outra requisição, em outra thread, pode ler ou sobrescrever.

Estereótipos: o papel de cada classe

Um **estereótipo** é uma anotação que transforma a classe em bean e declara a camada a que ela pertence. Todos derivam de `@Component`.

Anotação	Camada	O que acrescenta
`@Component`	qualquer	Só registra a classe como bean.
`@Service`	domínio	Nada além do nome: documenta que ali mora regra de negócio.
`@Repository`	dados	Traduz exceções do banco para a hierarquia `DataAccessException`.
`@RestController`	apresentação	`@Controller` + `@ResponseBody`: o retorno do método vira o corpo da resposta, em JSON.
`@Configuration` + `@Bean`	configuração	Cria, por um método, beans de classes que não são suas.

Usar `@Service` em vez de `@Component` não muda o comportamento, mas comunica intenção para quem lê e para a IA. No ServiceHub, os repositories são interfaces que estendem `JpaRepository`: o Spring Data gera a implementação sozinho (Semana 04).

Injeção de dependências pelo construtor

Injeção de dependências é a classe receber de fora os objetos de que precisa, em vez de criá-los com `new`. Há três formas no Spring: pelo construtor, por *setter* e direto no campo. A recomendada é a primeira.

No campo

```
@Service
class PedidoService {
  @Autowired
  private PedidoRepository repo;
}
```

Funciona, mas esconde a dependência, impede o `final` e obriga a subir o Spring para testar. Erro comum em tutoriais e em código gerado por IA.

No construtor

```
@Service
class PedidoService {
  private final PedidoRepository repo;
  PedidoService(PedidoRepository repo) {
    this.repo = repo;
  }
}
```

O campo é `final`, o objeto nunca nasce incompleto, e o teste cria a classe com `new`, passando um dublê. Com um único construtor, `@Autowired` é dispensável. Muitas dependências no construtor sinalizam uma classe que faz coisas demais.

7. Configuração externa

↑ conteúdo

O Spring Boot segue o princípio de **convenção sobre configuração**: olha o que está no *classpath* e configura o resto com padrões sensatos. Colocou o starter de web? Sobe um Tomcat embutido na porta 8080, com JSON pronto. Quer outro valor? Sobrescreva no `application.properties`.

Mas o mesmo `.jar` roda no seu notebook, na integração contínua e em produção. O que muda entre esses ambientes (porta, endereço do banco, senhas) não pode estar compilado: fica em **configuração externa**, ideia que o guia *The Twelve-Factor App* chama de "configuração no ambiente". O padrão do Spring é ler uma variável de ambiente com um valor de desenvolvimento como reserva:

```
server.port=${SERVER_PORT:8080}
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5433/servicehub}
#            ${VARIAVEL:padrao} → usa a variável; se não existir, o padrão de dev
```

Conceito	Como funciona
Precedência	Uma variável de ambiente vence o valor escrito no arquivo. `SERVER_PORT=9090` sobe a API na 9090 sem recompilar.
Perfis	`application-prod.properties` só vale quando o perfil `prod` está ativo, por exemplo com `SPRING_PROFILES_ACTIVE=prod`. Ele acrescenta ou sobrescreve valores do arquivo principal.
Segredos	Senha e chave nunca entram no Git. O arquivo `.env` fica no `.gitignore`; um `.env.example` sem valores reais é versionado para documentar as variáveis.

CONFUSÃO COMUM

Dar padrão a um segredo, como `${JWT_SECRET:minhachave}`, parece prático, mas, se alguém esquecer de definir a variável em produção, a aplicação sobe com a chave que está no Git. Para segredos, o padrão deve ser vazio, e a aplicação deve se recusar a subir sem eles.

8. Starters e o Initializr

↑ conteúdo

Um **starter** é uma dependência do Spring Boot que reúne a biblioteca de um recurso e a **autoconfiguração** dela. No Spring Boot 4, a autoconfiguração foi dividida em módulos por recurso, e cada recurso tem o seu starter:

Recurso	Starter no `pom.xml`	Semana
Web (Spring MVC)	`spring-boot-starter-webmvc`	3
JPA e Hibernate	`spring-boot-starter-data-jpa`	4
Migrações	`spring-boot-starter-flyway`	4
Validação	`spring-boot-starter-validation`	4
Saúde e métricas	`spring-boot-starter-actuator`	4
Segurança e JWT	`spring-boot-starter-security`, `spring-boot-starter-oauth2-resource-server`	8 e 9

Só a biblioteca

```
<groupId>org.flywaydb</groupId>
<artifactId>flyway-core</artifactId>
```

A aplicação sobe, as migrações não rodam, e nenhum erro aparece.

Com o starter

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-
flyway</artifactId>
```

A autoconfiguração entra junto e o Flyway roda na subida.

Regra para a vida toda do projeto: recurso que não ativa, sem log e sem erro? Procure primeiro o `spring-boot-starter-*` que está faltando.

O Spring Initializr, campo a campo

O **Spring Initializr** (`start.spring.io`) gera o esqueleto do projeto: `pom.xml`, Maven Wrapper, classe principal e teste de contexto.

Campo	Valor
Project · Language	Maven · Java
Spring Boot	4.x estável mais recente (o projeto de referência usa 4.1.1)
Group · Artifact	`com.servicehub` · `servicehub-api`
Package name	`com.servicehub` (corrija à mão)
Packaging · Java	Jar · 21
Dependências	Spring Web, Spring Boot DevTools e Lombok. O resto entra semana a semana.

CONFUSÃO COMUM

O hífen de `servicehub-api` não é válido em nome de pacote Java, então o Initializr sugere `com.servicehub.servicehub_api`. Corrija o *Package name* para `com.servicehub` antes de gerar. Depois de gerado, confira no `pom.xml` que a versão do Boot começa com 4, que `java.version` é 21 e que a dependência web é o `spring-boot-starter-webmvc`.

9. Git: controle de versão distribuído

↑ conteúdo

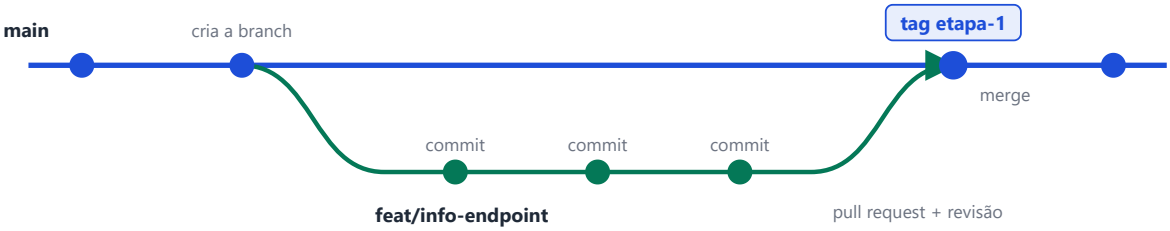
Um **sistema de controle de versão** guarda a história de cada mudança: quem fez, quando e por quê. No modelo **centralizado**, a história mora num servidor só, e quem está sem rede não registra nada. No modelo **distribuído**, como o Git, cada clone tem a história completa e funciona sem rede; a sincronização é feita depois.

Conceito	O que é
Commit	Uma fotografia do projeto, com autor, data, mensagem, um identificador (*hash*) e o commit anterior. A história é a corrente de commits.
Branch	Um ponteiro móvel para um commit. Criar uma é barato: é uma linha de trabalho isolada, não uma cópia da pasta. O `.gitignore` lista o que nunca entra no repositório, como `target/` e o `.env`.
Merge	Junta duas histórias. Se os dois lados mudaram as mesmas linhas, há **conflito**, e uma pessoa decide o resultado.
Remoto	Outra cópia do repositório, como o `origin` no GitHub. `push` envia commits; `fetch` e `pull` trazem.
Tag	Um nome fixo para um commit, como `etapa-1`. Diferente da branch, não se move.

10. GitHub Flow, commits e versões

↑ conteúdo

O **GitHub Flow** é um fluxo simples: a `main` está sempre funcionando; cada entrega nasce numa branch própria; a branch volta para a `main` por um **pull request** revisado.



Uma entrega no GitHub Flow: branch, commits pequenos, pull request revisado, merge e tag.

Ninguém faz `push` direto na `main`: no GitHub, uma regra de proteção da branch pode exigir pull request e aprovação. Se a branch quebrou algo, a `main` continua de pé e o problema aparece na revisão. O pull request é o momento do movimento **Revisar**: um colega confere o código, inclusive o que a IA gerou, contra a especificação. Um bom pull request tem título no padrão de commits, explica o que muda e como testar, e só entra depois de aprovado.

Conventional Commits

A especificação **Conventional Commits** padroniza a mensagem: `tipo: assunto`, com um escopo opcional entre parênteses, como `docs(alunos): ...`. O assunto é curto e diz o que muda; se precisar de mais, o corpo do commit explica o porquê. Na disciplina, as mensagens são em português.

Tipo	Quando	Exemplo
feat	Nova funcionalidade	`feat: prefixo /api/v1, endpoint de info e filtro X-Request-Id`
fix	Correção de bug	`fix: 404 em vez de 500 para serviço inexistente`
test	Só testes	`test: Testcontainers Postgres para os testes de contexto`
docs	Só documentação	`docs: README, ADR-0001 e CLAUDE.md da fase 0`
chore	Build e configuração	`chore: fixar Java 21`

Versionamento semântico

No **versionamento semântico** (*SemVer*), a versão tem a forma `MAIOR.MENOR.CORREÇÃO`, e cada parte é uma promessa para quem usa a API:

- **MAIOR** sobe numa mudança incompatível, em que o cliente precisa se adaptar (commit com `BREAKING CHANGE` ou `feat!`): 1.4.2 → 2.0.0.

- **MENOR** sobe numa funcionalidade nova e compatível (`feat`): 1.4.2 → 1.5.0.
- **CORREÇÃO** sobe numa correção compatível (`fix`): 1.4.2 → 1.4.3.

Numa API REST, remover um campo do JSON ou mudar o significado de um status é mudança MAIOR; é para isso que a rota carrega `/v1`, e uma quebra de contrato vira `/v2`. Versões `0.y.z` indicam desenvolvimento inicial, em que tudo pode mudar. O `0.0.1-SNAPSHOT` do `pom.xml` é isso, e `SNAPSHOT` é a convenção do Maven para versão em desenvolvimento.

PARA LEMBRAR

As tags da disciplina, `etapa-1`, `etapa-2` e assim por diante, marcam **entregas**, não versões semânticas. Use tag **anotada** (`git tag -a`), que guarda autor, data e mensagem, criada na `main` depois do merge. Na banca, a equipe apresenta a partir de uma tag, não "do que estava na minha máquina".

11. Contexto para a IA e ADRs

↑ conteúdo

O assistente de IA começa cada sessão sem memória do projeto. O que não estiver escrito, ele chuta, normalmente com a versão mais comum na internet, que não é a nossa. Gerenciar esse contexto é trabalho de engenharia, e ele mora em dois documentos versionados no repositório.

CLAUDE.md ou AGENTS.md

É um arquivo Markdown na raiz do projeto com o que um assistente precisa saber antes de mexer no código. O `CLAUDE.md` é lido pelo Claude Code ao abrir o projeto; o `AGENTS.md` é uma convenção aberta, lida por outros assistentes de código. A equipe escolhe um nome e mantém um só. O conteúdo deve ser curto, verdadeiro e atualizado a cada bloco; cada "pegadinha" nasce de um erro real:

```
# ServiceHub API
## Stack
Java 21 · Spring Boot 4.x · Maven Wrapper
## Comandos
mvnw.cmd clean install · mvnw.cmd spring-boot:run
## Convenções
- Rotas em /api/v1; POST responde 201 + Location
- Pacotes em com.servicehub; DTOs novos são record
- Commits em Conventional Commits, em português
## Pegadinhas
- Boot 4: recurso novo pede spring-boot-starter-<recurso>
- @MockBean não existe mais: use @MockitoBean
```

ADR: registro de decisão de arquitetura

Um **ADR** (*Architecture Decision Record*) é um texto curto que registra uma decisão de arquitetura, o contexto em que ela foi tomada e o que ela custa. Resolve um problema comum: meses depois ninguém lembra por que o sistema é assim, e a discussão recomeça do zero. O formato proposto por Michael Nygard tem cinco partes:

Seção	Responde
Título	Qual decisão, em poucas palavras, com número sequencial
Status	Proposto, aceito ou substituído (por qual ADR)
Contexto	Que problema e que forças existiam
Decisão	O que foi escolhido, na voz ativa: "usaremos..."
Consequências	O que melhora e o que piora, inclusive os custos

Escreva um ADR quando a escolha é cara de desfazer ou não é óbvia para quem chega. Um ADR nunca se apaga: se a equipe mudar de ideia, um ADR novo substitui o antigo, e a história fica. Para a IA, o ADR também é contexto: ela para de sugerir o que a equipe já descartou. Exemplo genérico, fora do ServiceHub:

```
# ADR-0003 — PostgreSQL em vez de banco de documentos
Status: aceito
## Contexto
Pedidos, clientes e produtos se relacionam e precisam de integridade
(um pedido não pode apontar para um cliente que não existe).
## Decisão
Usaremos PostgreSQL, com o schema versionado por migrações.
## Consequências
+ Chaves estrangeiras e transações garantem a integridade.
- Toda mudança de schema exige uma migração nova.
```

12. Aplicação no ServiceHub

↑ conteúdo

Trechos reais do projeto de referência; os cortes estão indicados.

A classe principal e os pacotes

A classe principal, `ServicehubApiApplication`, anotada com `@SpringBootApplication` e com o `main` que chama `SpringApplication.run`, fica no pacote raiz `com.servicehub`, para que a varredura encontre todos os subpacotes:

```
com.servicehub
├── ServicehubApiApplication.java
├── controller   rotas; delega ao service            (semana 3)
├── model/dto    records de entrada e saída          (semana 3)
├── service      regras de negócio                   (semana 4)
├── repository   acesso ao banco                     (semana 4)
├── model        entidades JPA                       (semana 4)
├── exception    erros em ProblemDetail              (semana 4)
├── config       filtros e OpenAPI
└── security     JWT e permissões                    (semanas 8 e 9)
```

O primeiro endpoint

Resumido, sem imports e sem as anotações OpenAPI da Semana 05:

```
@RestController
@RequestMapping("/api/v1/info")
public class InfoController {
    private final String version;

    public InfoController(ObjectProvider<BuildProperties> buildProperties) {
        BuildProperties props = buildProperties.getIfAvailable();
        this.version = props != null ? props.getVersion() : "dev";
    }

    @GetMapping
    public InfoResponse info() {
        return new InfoResponse("OK", version, "ServiceHub API operacional");
    }
}

public record InfoResponse(String status, String version, String message) { }
```

Repare: injeção pelo construtor, rota versionada e um `record` como DTO, sem Lombok. Na sua primeira versão, a versão pode ser um texto fixo, como `"0.0.1-SNAPSHOT"`. O `application.properties` usa `${VARIAVEL:padrao}` como na seção 7, e o segredo do JWT tem padrão vazio: `${JWT_SECRET:}`.

Histórico e tags

Três commits reais do projeto (`git log --oneline`, em ordem cronológica):

```
a59709a chore: fixar Java 21 e remover migracao solta fora de db/migration
ec38eae docs: README, ADR-0001 e CLAUDE.md da fase 0
61011c0 feat: prefixo /api/v1, endpoint de info e filtro X-Request-Id
```

O projeto tem uma tag anotada por fase (de `fase-0-baseline` a `fase-6-ci-observabilidade`), sete ADRs em `docs/adr/` (como `0001-stack-e-versoes.md`) e um `CLAUDE.md` que cresceu fase a fase; a pegadinha dos starters está escrita nele.

13. Construindo com IA

↑ conteúdo

Nesta semana a IA escreve o primeiro código. Sem contexto, é comum vir Spring Boot 3, rota sem versão e classe com Lombok. Com o `CLAUDE.md` e as decisões da equipe escritas, o resultado muda. Os quatro movimentos:

Movimento	O que você faz nesta semana
Especificar	Gere o projeto no Initializr, escreva o `CLAUDE.md` e a lista de decisões da equipe (stack e versões) antes de pedir código.
Gerar	Peça o plano, confirme, e só então o `InfoController`, o `InfoResponse` e o texto do ADR-0001.
Revisar	Boot 4 no pom? Starter webmvc? Pacote certo? Rota com `/api/v1`? Record em vez de Lombok? O ADR diz o que a equipe decidiu?
Provar	O `curl` devolve 200 com o JSON esperado, e a mudança chega à `main` por pull request revisado, com a tag `etapa-1`.

Prompt fraco e prompt bom

Fraco

```
cria um projeto spring boot com um endpoint de status
```

Sem versões, sem pacote, sem rota. A IA escolhe por você, em geral a versão mais comum nos exemplos da internet.

Bom

Diz versões e pacote, aponta o `CLAUDE.md`, fixa rota e formato da resposta, proíbe o que não queremos, pede plano antes e define o `curl` que prova. O texto completo está abaixo.

O prompt desta semana

Este prompt continua o da Semana 02: os documentos `docs/endpoints.md` e `docs/dominio.md` já estão no repositório.

```
Contexto: somos uma equipe construindo a API ServiceHub. Geramos o
projeto no start.spring.io: Maven, Java 21, Spring Boot 4.x,
Group com.servicehub, Artifact servicehub-api, Package com.servicehub,
dependências Spring Web, DevTools e Lombok. O CLAUDE.md na raiz tem
stack, comandos e convenções. docs/endpoints.md e docs/dominio.md
descrevem o contrato (todas as rotas em /api/v1).

Objetivo:
1. Conferir o pom.xml gerado e apontar qualquer divergência.
2. Criar os pacotes controller, service, repository, model, model/dto,
   exception, config e security dentro de com.servicehub.
3. Criar InfoController (com.servicehub.controller) com
   GET /api/v1/info devolvendo o record InfoResponse
   (com.servicehub.model.dto) com os campos status, version e message:
   {"status":"OK","version":"0.0.1-SNAPSHOT",
    "message":"ServiceHub API operacional"}
4. Escrever docs/adr/0001-stack-e-versoes.md no formato de Nygard
   (título, status, contexto, decisão, consequências) a partir das
   decisões da equipe listadas abaixo. Não acrescente decisões.

Restrições:
- Spring Boot 4.x e Java 21; nada de Boot 2 ou 3, nem javax.*.
- Recurso novo só com o spring-boot-starter-<recurso>; web é o
  spring-boot-starter-webmvc. Não adicione dependências agora.
- Injeção pelo construtor; nada de @Autowired em campo.
- DTO como record Java, sem Lombok.
- Não crie entidades, repositórios, segurança nem banco nesta etapa.

Antes de escrever código, mostre o plano: arquivos que vai criar ou
alterar, com o caminho completo. Espere eu confirmar.

Critério de aceite: mvnw.cmd spring-boot:run sobe sem erro, e
curl -i http://localhost:8080/api/v1/info responde HTTP 200 com
Content-Type application/json e o JSON acima.

Decisões da equipe para o ADR-0001:
<cole aqui a lista: versões e o motivo de cada uma>
```

O que a IA costuma errar nesta etapa

- **Versões antigas:** sugere Spring Boot 3 e Java 17, ou pacotes `javax.*`, porque viu mais exemplos assim.
- **Biblioteca sem starter:** `flyway-core` sozinho no pom; a migração não roda e nenhum erro aparece.
- **APIs que saíram:** `@MockBean` no lugar de `@MockitoBean`; Jackson 2 (`com.fasterxml.jackson`) onde o Boot 4 usa Jackson 3 (`tools.jackson`).
- **Um commit para tudo:** "update" com dezenas de arquivos, direto na `main`.
- **ADR inventado:** acrescenta decisões e motivos que a equipe não tomou.

Checklist de revisão

☐ `spring-boot-starter-parent` 4.x e `java.version` 21 no `pom.xml`?

☐ Classe principal em `com.servicehub`; controller em `com.servicehub.controller`?

☐ Rota exatamente `/api/v1/info`, com `InfoResponse` como record?

☐ Nenhum `@Autowired` em campo e nenhuma dependência nova sem pedido?

☐ O ADR-0001 tem as cinco seções e só as decisões da equipe?

☐ Commits pequenos, em Conventional Commits, numa branch, com pull request?

Prova

```
mvnw.cmd spring-boot:run          # Windows (./mvnw no Linux e macOS)
curl -i http://localhost:8080/api/v1/info
# HTTP/1.1 200 ... Content-Type: application/json
# {"status":"OK","version":"0.0.1-SNAPSHOT","message":"ServiceHub API operacional"}

git switch -c feat/info-endpoint
git add src/main/java/com/servicehub/controller/InfoController.java ...
git commit -m "feat: endpoint GET /api/v1/info"
git push -u origin feat/info-endpoint      # abre o pull request no GitHub
# depois da revisão e do merge:
git switch main && git pull
git tag -a etapa-1 -m "Etapa 1: setup do projeto" && git push origin etapa-1
```

Use `git add` com o caminho dos arquivos, e não `git add .`, para não commitar `target/` ou um `.env` por engano. Se a porta 8080 estiver ocupada, defina `SERVER_PORT`.

14. Glossário

↑ conteúdo

Framework	Estrutura pronta que chama o seu código na hora certa.
Inversão de controle	O framework, e não o seu programa, controla o fluxo e a criação de objetos.
Anotação	Metadado preso ao código, lido pelo compilador, por ferramentas ou pelo framework.
Reflexão	Capacidade de um programa inspecionar as próprias classes em execução.
Pacote	Agrupamento de classes que cria um espaço de nomes e limita a visibilidade.
Bean	Objeto criado e gerenciado pelo contêiner do Spring; singleton por padrão.
Estereótipo	Anotação que registra a classe como bean e indica sua camada.
Starter	Dependência do Boot que reúne a biblioteca e a autoconfiguração de um recurso.
Perfil	Conjunto de configurações ativado por ambiente, como `prod`.
Commit	Fotografia do projeto com autor, mensagem e ligação ao commit anterior.
Branch	Ponteiro móvel para um commit; uma linha de trabalho isolada.
Pull request	Pedido de revisão e merge de uma branch na `main`.
Tag anotada	Nome fixo para um commit, com autor, data e mensagem.
SemVer	Versão MAIOR.MENOR.CORREÇÃO, em que cada parte indica o tipo de mudança.
ADR	Registro curto de uma decisão de arquitetura: contexto, decisão, consequências.

15. Exercícios

↑ conteúdo

1. Explique com suas palavras a diferença entre biblioteca e framework e dê um exemplo de inversão de controle no ServiceHub.
2. Por que o Spring consegue ler `@RestController` em tempo de execução, mas não conseguiria ler uma anotação com `@Retention(RetentionPolicy.SOURCE)`?
3. Um colega criou `com.servicehub2.controller.PingController` e o endpoint responde 404, sem erro na inicialização. Qual é a causa provável e como corrigir?
4. Em qual camada você colocaria cada item, e por quê: (a) "ninguém contrata o próprio serviço"; (b) converter o JSON da requisição em DTO; (c) buscar os serviços ativos de uma categoria no banco?
5. O código abaixo tem um defeito grave numa API com vários usuários simultâneos. Qual é e por quê? `@Service class CarrinhoService { private List<Item> itensDoUsuario = new ArrayList<>(); }`

6. Reescreva com injeção pelo construtor e diga duas vantagens: `@RestController class AvaliacaoController { @Autowired private ReviewService service; }`
7. Com `server.port=${SERVER_PORT:8080}` no arquivo e a variável `SERVER_PORT=9090` definida no ambiente, em que porta a API sobe? E sem a variável?
8. Um colega adicionou `flyway-core` ao `pom.xml` e as migrações não rodam. O que você procura primeiro? Qual linha você acrescentaria ao `CLAUDE.md` para a IA não repetir o erro?
9. Escreva a mensagem de commit, em Conventional Commits, para: (a) corrigir um 500 que deveria ser 404; (b) adicionar o filtro de categoria na listagem de serviços; (c) atualizar o README.
10. A API está na versão 1.3.0. Qual é a próxima versão em cada caso: (a) corrigir um bug sem mudar o contrato; (b) adicionar um filtro novo, opcional; (c) renomear o campo `price` para `valor`?
11. Escreva um ADR curto, no formato de Nygard, para a decisão "rotas versionadas no caminho, em `/api/v1`".
12. Reescreva o prompt fraco "cria um projeto spring boot com um endpoint de status" seguindo o movimento Especificar, e liste três coisas que você conferiria na resposta.

16. Para estudar

↑ conteúdo

Leituras disponíveis na Minha Biblioteca, pelo acesso institucional:

- SCHILDT, H. *Java para iniciantes*. 6. ed. Bookman. Capítulo 8 (pacotes e interfaces) e capítulo 12 (anotações).
- FOWLER, M. *Padrões de arquitetura de aplicações corporativas*. Bookman, 2006. Capítulo 1 (camadas).
- MARTIN, R. C. *Arquitetura limpa*. Alta Books, 2019. Capítulos 22 e 32.
- MARTIN, R. C. *Código limpo*. Alta Books, 2009. Capítulos 2 e 3, úteis para revisar nomes e funções do código que a IA gera.
- MONTEIRO, E. R. et al. *DevOps*. Grupo A, 2021. Capítulos "Tipos de controles de versão" e "Criação de branches e commits com o Git".

Referências abertas:

- Documentação do Spring Boot: `docs.spring.io/spring-boot`. Conventional Commits: `conventionalcommits.org`. SemVer: `semver.org`. GitHub Flow: `docs.github.com`.
- NYGARD, M. *Documenting Architecture Decisions*, 2011. *The Twelve-Factor App*, fator "Config": `12factor.net`.